National Institute of Technology Karnataka, Surathkal
Department of Information Technology

IT 301 — Parallel Computing

RealFact

OpenMP-Accelerated, Independence-Aware Real-Time News Verification

Project Report

Adithya Shyam D	241IT005
Krishna	241IT039
P Suman	241IT049
Saurabh	241IT072

Primary reference: Z. Guo, M. Schlichtkrull and A. Vlachos, "A Survey on Automated Fact-Checking," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 178–206, 2022.

31 October 2026

Abstract

Misinformation spreads faster than people can check it. Fake-news classifiers that judge writing style fail when a false claim is written in a credible tone. Evidence-based verification avoids this: it retrieves related articles and reasons over them. But comparing a claim against a live pool of thousands of articles, in real time, is a large similarity-search problem. We present **RealFact**, a working prototype that verifies a news claim against a continuously refreshed corpus of live RSS articles. It has two contributions. The first is an *independence-aware* aggregation stage: it retrieves a wide candidate pool ($M = 10{,}000 \gg K$), clusters near-duplicate (syndicated) copies, and gives a cluster of n_c copies the weight $w_c = 1 + \lambda \log_2 n_c$ instead of n_c votes. It then issues a cited four-way verdict: Supported, Contradicted, Conflicting Evidence or Unverified. The second is an *exact* C++/OpenMP retrieval kernel, benchmarked across static, dynamic and guided scheduling at 1–16 threads and corpus sizes from 1.7K to 500K. On an Apple M3 Pro the kernel reaches a **4.61**× speedup over a non-OpenMP serial build at $N = 500$K (guided, 16 threads), sustaining about 81 GB/s of embedding reads. Every one of the 75 timed configurations returned article IDs bit-identical to the serial build. We then verified the system with a 12-check correctness suite and applied course techniques to the kernel. Restructuring the dot product for instruction-level parallelism made the serial scan **2.6**× faster and brought the best parallel time to 8.0 ms (96 GB/s, at the memory ceiling with 8 threads). Guided scheduling cut idle time on the heterogeneous cores from 11% to 1%, and multi-query blocking raised throughput **5**× by moving the kernel up the roofline. A controlled syndication test shows that naive duplicate counting and plain top-K retrieval flip a contested claim to *Contradicted*, while RealFact's verdict correctly stays *Conflicting Evidence*. We map each design decision to the lessons of Guo et al.'s survey, compare RealFact with existing approaches, and report a LIAR evaluation that exposes the dependence of accuracy on evidence-corpus coverage.

Contents

1 Introduction **3**
- 1.1 Problem . . . 3
- 1.2 Research question . . . 3
- 1.3 Contributions . . . 3

2 Background: What We Learned from the Survey **3**
- 2.1 The four-stage framework and our scope . . . 3
- 2.2 Lessons and the design decisions they led to . . . 4
- 2.3 Open challenges named in the survey . . . 5
- 2.4 Other related work . . . 5

3 Existing Approaches versus RealFact **5**

4 Datasets **6**

5 System Design **7**
- 5.1 Architecture . . . 7
- 5.2 Retrieval . . . 7
- 5.3 Retrieve-wide duplicate clustering . . . 7
- 5.4 Independence-weighted aggregation . . . 8
- 5.5 Parallelisation strategy . . . 8

6 Implementation **9**
- 6.1 Engineering issues encountered . . . 9
- 6.2 Relevance floor . . . 10
- 6.3 User interface . . . 10

7 Experimental Setup **10**

8 Results **10**
8.1 Parallel performance 10
8.2 Expected versus observed 12
8.3 Independence filter: controlled syndication test 12
8.4 Verification quality on LIAR 13
8.5 End-to-end latency 13

9 Verification and Course-Driven Optimisations **14**
9.1 Correctness test suite 14
9.2 Instruction-level parallelism in the dot product 14
9.3 Karp–Flatt metric 15
9.4 Load balance on heterogeneous cores 15
9.5 Chunk-size sweep 16
9.6 Weak scaling (Gustafson) 16
9.7 Roofline: multi-query cache blocking 16
9.8 Adaptive thread count and default schedule 17

10 Threats to Validity and Limitations **17**

11 Conclusion and Future Work **18**

A Reproducing the Results **19**

1 Introduction

1.1 Problem

Automated fact-checking asks whether a claim is supported by the available evidence. Two weaknesses of common approaches motivate this project:

1. **Style-based classifiers judge text, not truth.** A model trained to separate "fake" from "real" articles by wording fails when misleading content imitates the tone of credible reporting.
2. **Evidence-based systems are judged on accuracy alone.** Systems built on benchmarks such as FEVER, LIAR and HoVer have advanced retrieval and multi-hop reasoning. They are almost never evaluated for latency or multicore scalability, both of which matter when the evidence pool is live and large.

A third, less visible problem appears when the evidence pool is real news. Outlets republish the same wire report, so a ranked list of "evidence" can be ten copies of one story. If every copy counts as an independent confirmation, a single report can outvote several genuinely independent ones.

1.2 Research question

> *Can large-scale, multi-source evidence retrieval for real-time news verification run efficiently on a multicore CPU using OpenMP, without counting duplicate or syndicated reports as independent evidence?*

1.3 Contributions

1. An **independence-aware aggregation** stage: retrieve wide, cluster duplicates, keep the top-K clusters, and weight each by $w_c = 1 + \lambda \log_2 n_c$ (Section 5.4).
2. An **exact OpenMP similarity kernel** with thread-local, cache-line-padded heaps and deterministic tie-breaking, benchmarked across schedules × threads × corpus size (Sections 5.5 and 8.1).
3. Operation over **live, permitted news feeds**, ingested from 42 RSS feeds and refreshed every 20 minutes, instead of a frozen corpus.
4. An **explainable four-way verdict** that cites every evidence cluster with its weight and signed contribution, instead of a binary fake/real label.
5. A correctness suite and a set of course-driven kernel optimisations: ILP vectorisation, load-balance instrumentation, chunk tuning, weak scaling, Karp–Flatt analysis, and multi-query cache blocking (Section 9).
6. A web interface that runs the pipeline, shows how existing aggregation rules would judge the same evidence, and presents what we learned from the survey.

2 Background: What We Learned from the Survey

Our primary reference is the survey by Guo, Schlichtkrull and Vlachos [1]. It organises automated fact-checking into one framework and reviews datasets, methods and open problems. This section summarises the parts that shaped RealFact.

2.1 The four-stage framework and our scope

The survey decomposes automated fact-checking into four stages: *claim detection* (finding check-worthy claims), *evidence retrieval*, *verdict prediction*, and *justification production* (explaining the

verdict). Figure 1 places RealFact in that framework. Following our methodology, this project addresses evidence retrieval and verdict prediction. Claim detection is out of scope, since the user supplies the claim. Justification is covered only partially: we cite evidence and show each cluster's contribution, but do not generate free-text explanations.

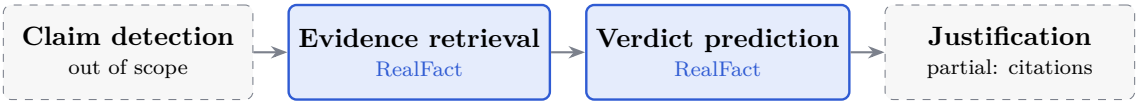


Figure 1: The automated fact-checking framework of Guo et al. [1], with RealFact's scope highlighted.

2.2 Lessons and the design decisions they led to

Table 1 lists each idea we took from the survey (paraphrased) and the RealFact decision it led to. The same mapping is shown in the application's "What we learned" tab, with a pointer to where each decision can be seen working.

Table 1: Lessons from the survey and how RealFact responds.

Survey point (paraphrased)	**RealFact design decision**
Fact-checking is a pipeline: retrieval of evidence followed by verdict prediction over that evidence.	Retrieval and verdict prediction are separate, separately timed stages. Verdicts come from evidence, not from the claim's writing style.
Claim-only datasets (e.g. LIAR) give no evidence documents, so models learn from surface cues and metadata.	Every verdict is computed from retrieved articles and lists them. With no relevant evidence the system answers *Unverified*.
Most systems assume a single trustworthy evidence source. Handling conflicting or untrustworthy evidence is an open problem.	*Conflicting Evidence* is a first-class verdict, and syndicated copies are discounted so one story cannot pose as many independent witnesses.
Fine-grained ratings such as "half-true" are usually meta-ratings over composite claims whose parts differ in veracity.	LIAR's middle band is reported separately and treated as closer to *Conflicting* than to *Neutral*.
"Not enough information" (FEVER's NEI) is a legitimate outcome.	*Unverified* is triggered by an explicit sufficiency threshold θ on polar evidence mass, plus a relevance floor.
Datasets have artefacts: artificially written claims let models exploit claim-only cues, and web-retrieved evidence can leak the answer.	Results are planned on FEVER, LIAR and FakeNewsNet side by side. The engine's `exclude` mask removes a claim's own article before the scan.
Benchmarks are built on static snapshots, while real claims concern recent events.	Evidence comes from 42 permitted live RSS feeds, re-ingested on a timer, with timestamps and source domains kept.
Surveyed systems are evaluated on accuracy, F1 and evidence recall. Efficiency is rarely reported.	A second, independently falsifiable axis: an OpenMP kernel measured across schedules, threads and corpus sizes, checked bit-for-bit against serial.

2.3 Open challenges named in the survey

Table 2: Open challenges discussed in the survey and RealFact's position.

Challenge	Addressed?	How / why not
Conflicting or untrustworthy evidence	**Yes**	Conflicting verdict ($\delta < D < 1 - \delta$); independence weighting.
Label choice for fine-grained ratings	**Yes**	LIAR middle band reported separately.
Dataset artefacts and claim-only cues	Partial	Multiple datasets reported; listed as a threat to validity.
Evidence leakage	**Yes**	Leakage mask in the engine; source article excluded.
Up-to-date evidence	**Yes**	Live RSS ingestion every 20 minutes.
Explainability / faithful justification	Partial	Cited clusters with $w_c p_c \sigma_c$; no generated text.
Multimodal and multilingual claims	No	English text only.
Efficiency and scalability	**Yes**	This project's systems contribution.

2.4 Other related work

FEVER [2] established the retrieve-then-classify formulation with SUPPORTED / REFUTED / NOT ENOUGH INFO labels over Wikipedia. **LIAR** [3] provides 12,836 short political statements on a six-point truth scale with speaker metadata but no evidence documents. **FakeNewsNet** [4] provides full news articles with source URLs and is the only one of the three that contains syndicated copies. **HoVer** and **IMRRF** extend retrieval to multi-hop reasoning. Like the others, they report accuracy but not multicore latency. We use **BGE-small-en-v1.5** [5] for embeddings, **OpenMP 5.2** [6] for parallelism, and the **Roofline** model [7] to predict where speedup flattens.

3 Existing Approaches versus RealFact

Table 3: Comparison with existing approaches. The last row is this project.

System	Evidence	Up to date	Retrieval	Duplicate handling	Verdicts	Conflicting evidence	Parallel perf. reported
Style-based classifiers	None (style)	—	None	—	Binary	No	No
LIAR baselines	Claim + metadata	Static	None	—	6-way	No	No
FEVER pipeline	Wikipedia	Static dump	Doc + sentence top-k	None	3-way	Folded into NEI	Accuracy only
HoVer / IMRRF	Wikipedia, multi-hop	Static dump	Iterative multi-hop	None	2–3-way	No	Accuracy only
RealFact	Live multi-outlet news	**Yes** (RSS)	Exact OpenMP scan, top-M	Cluster at τ, $1 + \lambda \log_2 n$	4-way, cited	**Yes** (first-class)	**Yes** (speedup, E_p, p95, Wilcoxon)

Four differences can be observed directly in the running application:

1. **Retrieve wide, then cluster.** Existing pipelines keep the top-K articles. If ten outlets syndicate one report and $K = 10$, those slots hold one story, and clustering after truncation leaves nine empty. RealFact clusters the $M = 10{,}000$ pool first and then keeps the top-K *clusters*.

2. **Independence weighting.** n copies count as $1 + \lambda \log_2 n$ votes. With $\lambda = 0.5$, 16 copies count as 3 votes, not 16 (Figure 2).
3. **Four-way verdict.** Disagreement between independent sources is reported, not forced into a binary label.
4. **Measured parallel retrieval.** The scan is timed on a disclosed platform and must return bit-identical IDs to a serial build.

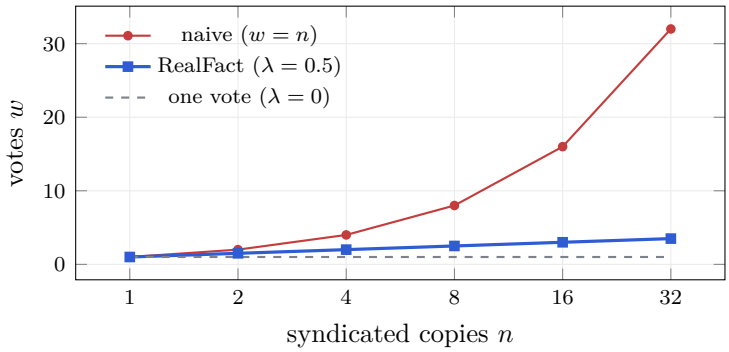


Figure 2: Cluster weight as a function of the number of syndicated copies.

4 Datasets

Table 4: Datasets and their roles.

Dataset	Role	Size	Labels mapped to 3-way
FEVER	Verification quality (train/tune)	185,445 claims (145,449 / 19,998 / 19,998)	SUPPORTED→SUPPORT, REFUTED→CONTRADICT, NEI→NEUTRAL
LIAR	Generalisation to short political claims	12,836 statements, 80/10/10	true, mostly-true→SUPPORT; false, pants-fire→CONTRADICT; half-true, barely-true→NEUTRAL
FakeNewsNet	Real-world news; only source of syndication	23,196 items, 70/10/20 grouped by duplicate cluster	Headline as claim; real→SUPPORT, fake→CONTRADICT; also merged not-supported
Live corpus	Unlabelled evidence pool searched by the engine	1,759 articles from 21 outlets at report time; synthetic up to 500K for scaling	Title, lead, timestamp, source domain

Label harmonisation. System verdicts map to the three-way benchmark labels as Supported → SUPPORT, Contradicted → CONTRADICT, and both Unverified and Conflicting Evidence → NEUTRAL. The two are also reported separately, because collapsing them hides the behaviour the independence filter is meant to produce.

Preprocessing. HTML and boilerplate are stripped, whitespace is normalised, and non-English items are dropped. Each article is hashed on (source domain, normalised headline) as a cheap exact-duplicate filter before embedding. Claims and articles are encoded with BGE-small-en-v1.5 into 384-dimensional unit vectors. Article embeddings are computed once at ingestion and cached. RSS supplies a headline and a lead paragraph only; we never scrape article pages. Because BGE truncates at 512 tokens, an article is represented by its lead in any case, which also helps syndication detection, since syndicated copies share leads.

Leakage control. For any claim drawn from a labelled benchmark, the source article and any article sharing its deduplication key are excluded from the scan through the engine's exclusion mask.

5 System Design

5.1 Architecture

Figure 3 shows the end-to-end pipeline. Phase 1 builds the evidence index at ingestion time. Phase 2 verifies a claim. A separate benchmarking arm times the retrieval kernel only, so the systems claim and the accuracy claim can be falsified independently.

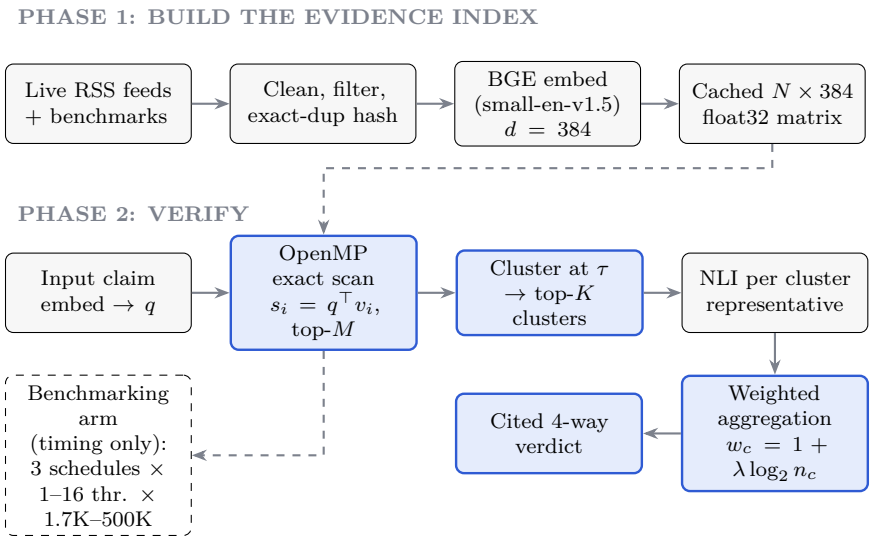


Figure 3: RealFact pipeline. Blue stages are this project's novel contributions.

5.2 Retrieval

Let the corpus be $\{a_i\}_{i=1}^{N}$ with cached unit-normalised embeddings $v_i \in \mathbb{R}^{384}$, and let the claim embed to q (BGE's retrieval instruction prefix is prepended to the claim). Because all vectors are unit-normalised, cosine similarity reduces to a dot product:

$$s_i = \cos(q, v_i) = q^\top v_i. \tag{1}$$

The scan is exhaustive and exact. We deliberately use brute force rather than an approximate (ANN) index, because an ANN index would change *what* is retrieved between configurations and mix a recall change into the speedup measurement. Candidates below a relevance floor (cosine < 0.68 by default) are not treated as evidence (Section 6.2).

5.3 Retrieve-wide duplicate clustering

The scan returns the top M candidates with $M \gg K$ (default $M = 10{,}000$, $K = 10$). These are clustered by single-pass leader clustering in score order (Algorithm 1). Only then are the top-K clusters kept, and each cluster's highest-scoring member becomes its representative. The order of operations is the important detail: clustering after truncating to K would let copies of one story fill all K slots.

Algorithm 1 Retrieve-wide greedy clustering

Require: candidate vectors $V_1, \ldots, V_m$ sorted by score (best first), threshold τ, cluster budget K

$L \leftarrow [\,]$ ▷ leaders
for $j = 1$ **to** m **do**
  if $L \neq \emptyset$ **and** $\max_{\ell \in L} V_\ell^\top V_j > \tau$ **then**
    assign j to $\arg\max_{\ell \in L} V_\ell^\top V_j$
  else if $|L| < K$ **then**
    $L \leftarrow L \cup \{j\}$; open a new cluster led by j
  else
    leave j unassigned ▷ belongs to a story outside the top K
  end if
end for
return clusters with sizes n_c (after the K leaders are found, the remaining assignment is one matrix product)

5.4 Independence-weighted aggregation

A 3-way NLI cross-encoder labels each cluster representative r_c against the claim, with the article as premise and the claim as hypothesis. This gives a signed stance $\sigma_c \in \{+1, -1, 0\}$ for SUPPORT-/CONTRADICT/NEUTRAL and a confidence $p_c \in [0, 1]$. A cluster of n_c syndicated copies is not n_c independent witnesses, and not exactly one either, so its weight interpolates between the two:

$$w_c = 1 + \lambda \log_2 n_c, \qquad \lambda \in [0, 1]. \tag{2}$$

The net signed stance S and the supporting share of non-neutral weight D are

$$S = \frac{\sum_c w_c p_c \sigma_c}{\sum_c w_c p_c}, \qquad D = \frac{\sum_{c:\sigma_c = +1} w_c p_c}{\sum_{c:\sigma_c \neq 0} w_c p_c}. \tag{3}$$

Table 5: Four-way verdict rule (conditions checked top to bottom).

Condition	Verdict
$\sum_{c:\sigma_c \neq 0} w_c p_c < \theta$	Unverified
$\delta < D < 1 - \delta$	Conflicting Evidence
$S \geq 0$ otherwise	Supported
$S < 0$ otherwise	Contradicted

Defaults: $\tau = 0.90$, $\lambda = 0.5$, $\theta = 0.6$, $\delta = 0.25$, relevance floor 0.68. All are adjustable from the user interface.

5.5 Parallelisation strategy

The scan is parallelised with `#pragma omp parallel for schedule(runtime)` over the corpus index, so the schedule kind and chunk size are run-time parameters. Each thread keeps a private top-M min-heap, aligned and padded to a 64-byte cache line to prevent false sharing on heap metadata. Embeddings are one contiguous row-major $N \times 384$ float32 array, so each thread's chunk is a unit-stride stream. Ties are broken by index, which makes the result a strict total order and the returned IDs deterministic.

Listing 1: Core of the parallel kernel (`engine/realfact_engine.cpp`).

```
struct alignas(64) LocalHeap { std::vector<Hit> h; /* + padding */ };

#pragma omp parallel num_threads(threads)
{
    LocalHeap& mine = heaps[omp_get_thread_num()];
```

```
    #pragma omp for schedule(runtime) nowait
    for (int64_t i = 0; i < N; ++i) {
        if (exclude && exclude[i]) continue; // leakage control
        mine.push(Hit{dot(q, emb + i * d, d), i}, M); // thread-local top-M
    }
}
// Serial reduction, independent of N: concatenate P heaps, nth_element, sort.
```

The same source file is compiled twice: once with OpenMP (`librealfact_omp`) and once without (`librealfact_serial`). The serial build is the reference T_1, so OpenMP runtime overhead is not charged to the baseline.

Merge refinement. The methodology specified a heap merge of the P local heaps, costing $O(PM \log M)$. At 16 threads and $M = 10{,}000$ this serial step (160K heap pushes) limited speedup, as Amdahl's law predicts. We replaced it with concatenate + `nth_element` + sort, costing $O(PM + M \log M)$. Because the comparison is a strict total order, the output is identical. At $N = 500$K, guided with 16 threads went from 11.46 ms to 9.48 ms median.

6 Implementation

Table 6: Code layout.

File	Responsibility
`engine/realfact_engine.cpp`	Exact top-M scan; OpenMP and serial builds; C ABI used through `ctypes`.
`backend/ingest.py`	Phase 1: fetch 42 RSS feeds in parallel, clean, filter, hash, embed, append.
`backend/store.py`	SQLite metadata + contiguous embedding file; row i = article id i.
`backend/models.py`	BGE-small-en-v1.5 embedder; NLI cross-encoder (`nli-deberta-v3-xsmall`).
`backend/pipeline.py`	Phase 2, ablations, and the syndication stress test.
`backend/bench.py`	Benchmarking arm, platform disclosure, exact Wilcoxon signed-rank test.
`eval/run_eval.py`	FEVER / LIAR / FakeNewsNet harness with label mapping and leakage control.
`backend/app.py`, `web/`	Flask API and the single-page interface (seven tabs).

6.1 Engineering issues encountered

- **OpenMP on macOS.** Apple clang ships no OpenMP runtime. We link against a prebuilt LLVM libomp 19.1.5 in `third_party/libomp` (libomp 17 lacked a symbol clang 21 emits).
- **Two OpenMP runtimes in one process.** PyTorch ships its own libomp. If PyTorch initialises first, our runtime crashes on its first parallel region. The engine libraries are therefore loaded at package import, before any PyTorch import.
- **Benchmark isolation.** An early sweep overlapped with feed ingestion (embedding with PyTorch) and showed CPU utilisation above 100% for one thread. Benchmarks and ingestion now share a lock and never run together.
- **Thread pinning.** macOS exposes no thread-affinity API, so `OMP_PROC_BIND` and `OMP_PLACES` have no effect. We disclose this instead of claiming pinned results.

6.2 Relevance floor

While testing we found that NLI models label off-topic premises as "contradiction". For the claim *"The Earth is flat"*, an article about a Martian meteorite was labelled CONTRADICT with $p = 1.00$. A larger model trained on MNLI, FEVER and ANLI showed the same behaviour, so the cause is input relevance, not model size. We therefore gate evidence by a relevance floor before NLI. With the floor at 0.68 the highest-scoring article for that claim (cosine 0.649) is excluded, and the verdict becomes *Unverified*, which is correct for a news corpus that does not discuss the claim.

6.3 User interface

The interface has seven tabs: *Verify a claim* (verdict, S and D meter, per-stage timings, an existing-vs-RealFact verdict table, and cited evidence cards with a "Simulate syndication ×10" button); *What we learned (survey)*; *Existing vs RealFact*; *Methodology*; *Parallel benchmark* (run sweeps, charts, expected-vs-observed table, platform disclosure); *Live corpus*; and *Evaluation*.

7 Experimental Setup

Table 7: Platform disclosure.

CPU	Apple M3 Pro, 11 cores (5 performance + 6 efficiency), no SMT
Caches	L1d 64 KB; L2 16 MB (performance cluster), 4 MB (efficiency cluster)
Memory	18 GB unified (19,327,352,832 bytes)
Compiler	Apple clang 21.0.0, `-O3 -std=c++17 -ffp-contract=off`; serial build without `-fopenmp`
OpenMP runtime	LLVM libomp 19.1.5; max 11 threads
Pinning	Not available on macOS (`OMP_PROC_BIND`/`OMP_PLACES` unset and not honoured)
Models	BAAI/bge-small-en-v1.5 (384-d); cross-encoder/nli-deberta-v3-xsmall; CPU inference

Protocol. Schedules static, dynamic and guided (default chunk) at 1, 2, 4, 8 and 16 threads, on the live corpus ($N = 1{,}725$ at benchmark time) and synthetic corpora of 10K, 50K, 100K and 500K random unit vectors. Retrieval cost depends only on $N \times d$, not on the text, so synthetic vectors are valid for timing. $M = 10{,}000$ (or N if smaller). Each configuration runs 2 discarded warm-ups and 11 timed queries; we report the median, IQR and p95. Queries are the same 11 per configuration, so latencies are paired, and schedules are compared with an exact two-sided Wilcoxon signed-rank test. For every query in every configuration, the returned IDs are compared with the serial build's.

Roofline expectation. The kernel performs 2 flops per 4-byte element loaded (0.5 flop/byte), placing it in the memory-bound region of the roofline. The working set at $N = 500$K is $500{,}000 \times 384 \times 4\,\text{B} = 768\,\text{MB}$, far beyond any cache.

8 Results

8.1 Parallel performance

This subsection reports the original kernel (*v1*). Section 9 reports the optimised kernel (*v2*) and compares the two.

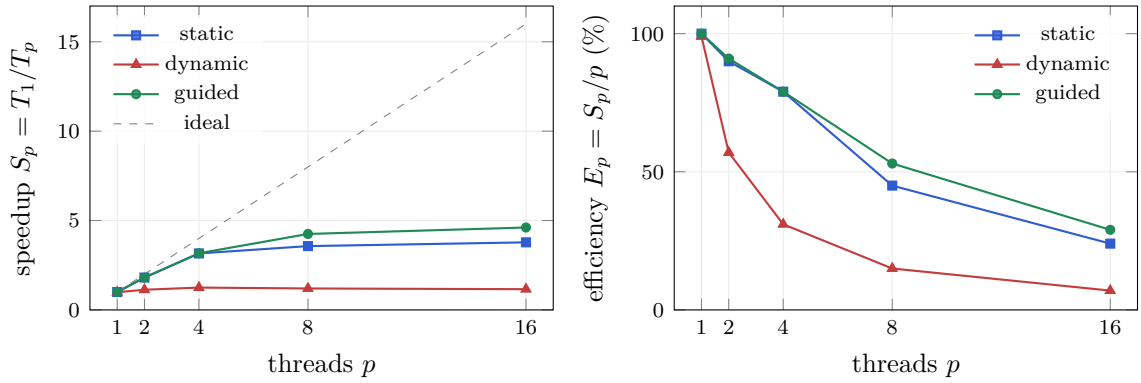


Figure 4: Speedup and parallel efficiency at $N = 500{,}000$ ($T_1 = 43.68$ ms, non-OpenMP serial build).

Table 8: Retrieval kernel at $N = 500{,}000$, $M = 10{,}000$. Median and IQR over 11 runs. * more threads than physical cores.

Schedule	p	Median ms	IQR	p95	S_p	E_p	q/s	IDs = serial
serial (no OpenMP)	1	43.677	0.528	—	1.00	—	23	—
static	1	43.758	2.136	48.587	1.00	100%	23	✓
static	2	24.190	0.085	24.414	1.81	90%	41	✓
static	4	13.811	0.137	13.987	3.16	79%	72	✓
static	8	12.233	0.459	13.347	3.57	45%	82	✓
static	16*	11.553	1.043	14.341	3.78	24%	87	✓
dynamic	1	43.998	0.148	44.302	0.99	99%	23	✓
dynamic	2	38.555	0.396	39.330	1.13	57%	26	✓
dynamic	4	34.986	0.241	35.671	1.25	31%	29	✓
dynamic	8	36.304	0.187	36.554	1.20	15%	28	✓
dynamic	16*	37.609	0.199	38.061	1.16	7%	27	✓
guided	1	43.878	0.257	44.072	1.00	100%	23	✓
guided	2	24.081	0.087	24.242	1.81	91%	42	✓
guided	4	13.793	0.145	14.337	3.17	79%	73	✓
guided	8	10.289	0.354	11.332	4.25	53%	97	✓
guided	16*	**9.481**	0.401	9.991	**4.61**	29%	**105**	✓

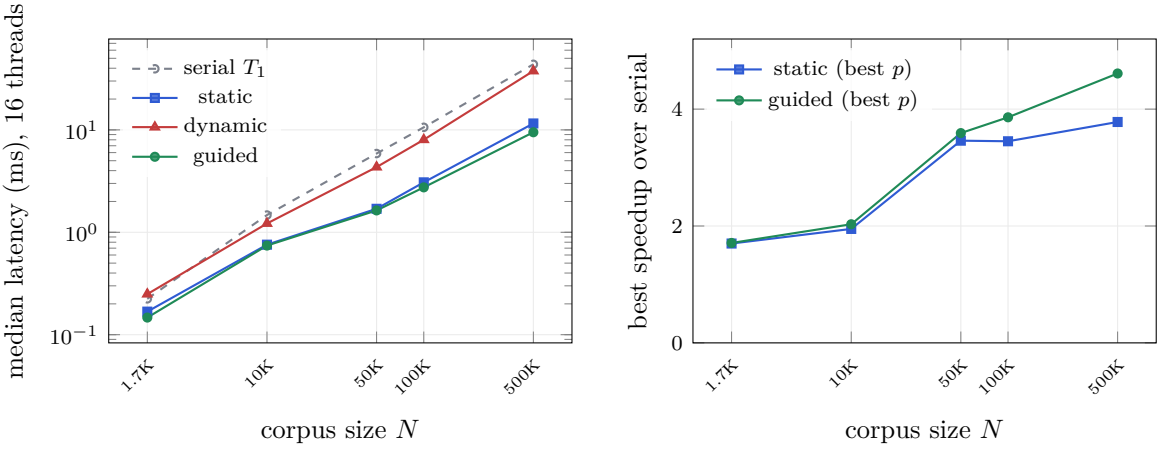


Figure 5: Corpus scaling. Left: median latency at 16 threads (log–log). Right: best speedup per schedule at each corpus size.

Table 9: Exact Wilcoxon signed-rank tests on paired per-query latencies at 16 threads ($n = 11$). With $n = 11$ the smallest attainable two-sided p is 9.8×10^{-4}. $r = |z|/\sqrt{n}$.

	static vs dynamic		**static vs guided**	
Corpus	p	r	p	r
live (1,725)	9.8×10^{-4}	0.88	0.083	0.54
10K	9.8×10^{-4}	0.88	0.147	0.46
50K	9.8×10^{-4}	0.88	1.000	0.00
100K	9.8×10^{-4}	0.88	0.0068	0.78
500K	9.8×10^{-4}	0.88	9.8×10^{-4}	0.88

Correctness. Across all 5 corpus sizes × 15 configurations × 11 queries, the returned article IDs were bit-identical to the serial build in every case. Parallelisation changes how retrieval is computed, not what is retrieved.

OpenMP entry cost. The 1-thread OpenMP build minus the non-OpenMP build was +0.024 ms on the live corpus and within run-to-run noise (−0.43 to +0.20 ms) at larger sizes.

8.2 Expected versus observed

Table 10: Predictions from the methodology checked against measurements.

Prediction	**Observed**
Near-linear speedup at 1–4 threads, then flattening at the memory-bandwidth ceiling before cores run out.	**Confirmed.** 79% efficiency at 4 threads, then gains slow sharply. At 16 threads (guided) the kernel reads 768 MB in 9.48 ms, about 81 GB/s, against 17.6 GB/s single-threaded. The flattening begins before all 11 cores are used, as the 0.5 flop/byte roofline predicts.
Static wins, because every article costs the same 384-d dot product.	**Not confirmed on this machine.** Guided beats static at 8 and 16 threads ($p = 9.8 \times 10^{-4}$ at 500K). The methodology anticipated this case: a dynamic or guided advantage indicates an external imbalance source. On the M3 Pro that source is the mix of 5 performance and 6 efficiency cores. Static gives every core an equal share, so the slow cores finish last. Guided rebalances. Dynamic with the default chunk of 1 pays an atomic counter increment per article and is the slowest by a wide margin.
A visible step where the working set leaves the cache.	Throughput per article drops between 10K (15 MB, within the 16 MB L2) and 50K (77 MB). Above that, scaling is close to linear in N.
OpenMP entry cost is small and charged separately.	Confirmed: about 0.02 ms, within noise at larger corpora.
Small corpora gain little.	Confirmed: at $N = 1{,}725$ the scan takes 0.22 ms serially and the best speedup is 1.7×. Thread start-up is comparable to the work.

8.3 Independence filter: controlled syndication test

To exercise the independence filter on a real claim, the application can republish one real article as n near-identical copies (cosine ≈ 0.99) under clearly labelled simulated outlets, for a single query. The copies are appended to the matrix that the OpenMP kernel scans, so the whole pipeline runs on them; nothing is written to the stored corpus.

Claim: *"Kuwait-flagged tanker with 21 Indian seafarers hit near Strait of Hormuz, crew safe".* Without syndication, four independent clusters were retrieved: one SUPPORT (indianexpress.com, $p = 0.97$), two NEUTRAL (cnbc.com), and one CONTRADICT (euronews.com, $p = 0.80$, reporting three tankers on fire). The verdict was *Conflicting Evidence* ($S = 0.04$, $D = 0.55$). We then republished the contradicting euronews.com article $\times 10$.

Table 11: Same retrieval and NLI after syndicating the contradicting article $\times 10$; only the aggregation rule differs.

Aggregation rule	Verdict	S	D	Votes $\sum w$
RealFact, log-weighted ($\lambda = 0.5$)	**Conflicting Evidence**	-0.22	0.31	6.2
One vote per cluster ($\lambda = 0$)	Conflicting Evidence	0.04	0.55	4.0
Naive duplicate counting	Contradicted	-0.62	0.10	15.0
Filter off: raw top-K articles	Contradicted	-0.56	0.14	10.0

One euronews.com report, copied ten times, outvotes the independent supporting report under naive counting and under plain top-K retrieval, and both flip to *Contradicted*. RealFact counts the 11-article cluster as $1 + 0.5 \log_2 11 = 2.73$ votes. The contradicting side moves D from 0.55 to 0.31, which is still inside the disagreement band, so the verdict remains *Conflicting Evidence*. The retrieve-wide rule also matters: in the same run, truncating to $K = 10$ before clustering would have kept only 4 independent stories and wasted 6 of the 10 slots on copies.

On unmodified live data the four rules agreed on every claim we tested, because cross-outlet reports of the same event score around 0.75–0.88 and stay below $\tau = 0.90$. They are independent reports, not syndicated copies. This is the expected control result: a filter that changed verdicts without syndication would be removing genuine evidence.

8.4 Verification quality on LIAR

We ran the evaluation harness on the official LIAR test split. 1,267 of its 1,283 statements carry one of the six labels. Each claim was verified against the live corpus (1,725 articles) with leakage control.

Table 12: LIAR test split (3-way mapping). All four aggregation rules produced identical results.

Mode	Accuracy	Macro F1	F1 SUP	F1 CON	F1 NEU
RealFact (any aggregation rule)	37.6%	0.188	0.000	0.017	0.546
Majority class (always NEUTRAL)	37.6%	—	—	—	—

Verdicts were 1,255 Unverified (99.1%) and 12 Contradicted. On the middle band (half-true, barely-true) they were 473 Unverified and 4 Contradicted. Median end-to-end latency was 7.8 ms per claim (p95 22.5 ms). It is low because NLI runs only when evidence clears the relevance floor.

Interpretation. LIAR's statements date from 2007–2016, while the evidence pool is a few days of current news. Almost no claim has relevant evidence, so the system correctly answers *Unverified* instead of guessing, and accuracy equals the majority-class baseline. This run measures corpus coverage, not reasoning quality. The methodology expected LIAR to score lowest. The identical results across aggregation rules are the expected control, since LIAR contains no syndicated copies. A meaningful accuracy comparison needs an evidence pool matched to the dataset: FakeNewsNet's articles, or the FEVER Wikipedia dump (about 5.4M documents) indexed into the same engine.

8.5 End-to-end latency

For a typical claim on the live corpus (*"Ukraine says at least 13 killed in Russian attack on train station"*), stage times were: claim embedding 8.1 ms, OpenMP scan 0.2 ms, clustering 0.1 ms, NLI

on the representatives 89.3 ms, aggregation 0.1 ms, and 98.2 ms in total. At the current corpus size NLI dominates. The scan becomes the significant cost at hundreds of thousands of articles, where it reaches 9.5–44 ms per query.

9 Verification and Course-Driven Optimisations

This section first verifies the system's correctness. It then improves the retrieval kernel with techniques from the Parallel Computing course and measures each change. All v2 numbers come from the benchmark run finished at 2026-10-31 18:53:49, with the same protocol as Section 7.

9.1 Correctness test suite

Table 13: Correctness checks (`tests/test_correctness.py`; also runnable from the Benchmark tab). All 12 pass on v1 and v2.

Check	What it verifies
Engine vs NumPy	45 schedule × thread × chunk configurations return the exact top-M against a float64 reference.
Bit-exactness	36 parallel runs at $M = 10{,}000$: IDs and scores identical to the non-OpenMP build.
Exact ties	A 40-way tie (duplicated rows) comes back in ascending index order under every schedule.
Edge cases	$M > N$, $M = 1$, $N = 1$, $N = 0$, more threads than rows, and $d \in \{1, 3, 17, 383, 385\}$ (remainder loops).
Leakage mask	1,003 masked rows are never returned, and the result equals the top-M of the unmasked rows.
Batched kernel	The multi-query kernel returns, for each query, exactly the single-query result.
Row accounting	Per-thread row counts sum to N under every schedule: no row is skipped or scanned twice.
Aggregation	S, D and the verdict table on hand-computed cases, including the syndication case study.
Clustering	Leader clustering groups copies, respects K, and leaves stories beyond K unassigned.
Wilcoxon test	Exact p-values equal brute-force enumeration of all 2^n sign assignments.
Ingestion	The duplicate key ignores case and punctuation but not the outlet; the English filter works.
Label maps	LIAR and FEVER mappings match methodology §I-B.

The suite found no defects in the v1 code. One input-validation weakness was fixed: a number field sending "10.0" for K used to raise an error.

9.2 Instruction-level parallelism in the dot product

Disassembling v1 showed that clang had vectorised the dot product into 4-lane NEON multiplies, but every row accumulated into a *single* vector register. Each of the 96 vector additions per 384-d row had to wait for the previous one, a loop-carried dependency, so one core ran at floating-point-add *latency*, not throughput. That is why v1's serial scan read only 17.6 GB/s.

v2 keeps 16 independent partial sums, which compile to four independent 4-lane registers, and reduces them with a fixed tree. The summation order is written out explicitly, so no `-ffast-math` reassociation is needed and the serial and OpenMP builds still agree bit for bit. We avoided `#pragma omp simd reduction` for the same reason: it lets the compiler choose the reduction order, and the order could then differ between the two builds. We also tried FMA contraction

(`-ffp-contract=on`). It stayed bit-exact but gave no measurable gain, because the kernel is memory-bound at that point, so we kept it off.

Table 14: Kernel v1 → v2 at $N = 500{,}000$, $M = 10{,}000$ (medians of 11 runs).

	v1	**v2**	**change**
Serial T_1 (no OpenMP)	43.68 ms	16.82 ms	2.60× faster
Single-core bandwidth	17.6 GB/s	45.7 GB/s	
Best parallel time	9.48 ms (guided, 16T)	8.04 ms (static, 8T)	1.18× faster
Bandwidth at best	81 GB/s	96 GB/s	
Speedup over own T_1	4.61×	2.09×	lower
Threads to reach 90% of best	16	4	

A second v1 run (18:17) gave $T_1 = 41.7$ ms and best 10.5 ms, an indication of run-to-run variance.

Why speedup fell while time improved. The faster core reaches the shared memory-bandwidth ceiling (about 96 GB/s) with only 4 threads, so additional threads have nothing left to gain. This is the roofline in practice: optimising the serial code raised T_1's performance, so speedup over T_1 shrank. Speedup only means something next to the baseline it is measured against, and we report both.

9.3 Karp–Flatt metric

The experimentally determined serial fraction $e = (1/S_p - 1/p)/(1 - 1/p)$ separates a fixed serial part (constant e, Amdahl) from overhead that grows with p (rising e).

Table 15: Speedup and Karp–Flatt e at $N = 500{,}000$.

	v1 guided		**v2 guided**		**v2 static**	
p	S_p	e	S_p	e	S_p	e
2	1.81	0.105	1.56	0.278	1.57	0.271
4	3.17	0.087	1.82	0.400	2.03	0.323
8	4.25	0.126	1.85	0.476	2.09	0.403
16	4.61	0.165	1.86	0.508	1.92	0.489

In v2, e rises steadily with p. The limit is therefore a contended shared resource (memory bandwidth), not a fixed serial section. The only serial code is the P-heap merge, which costs about 0.1 ms. v1's e stays small because its compute-bound core left bandwidth to spare.

9.4 Load balance on heterogeneous cores

v2 times each thread inside the worksharing loop (`rf_topm_stats`), using cache-line-padded per-thread counters so that the instrumentation cannot cause false sharing.

Table 16: Per-thread load balance at 11 threads (5 performance + 6 efficiency cores), $N = 500{,}000$.

Schedule	**Kernel ms**	**max/mean busy**	**Idle**	**Rows per thread**
static	9.14	1.12	11%	45,454–45,455
dynamic (chunk 1)	39.99	1.01	1%	40,489–51,814
dynamic (chunk 4096)	8.74	1.02	2%	20,480–69,632
guided	**8.15**	1.01	1%	31,554–58,761

Static gives every thread an equal share of rows, so the threads that land on slower cores finish last and the rest sit idle for 11% of the loop. Guided and chunked dynamic let faster cores take

up to 1.9× more rows (last column), so all threads finish within 1–2% of one another. Dynamic with chunk 1 is perfectly balanced but 4.9× slower: it pays an atomic counter increment for every 384-float row, and that overhead grows relative to the work once the row itself becomes cheaper (v2).

9.5 Chunk-size sweep

Table 17: Median kernel time (ms) by chunk size at 11 threads, $N = 500{,}000$. Static: 8.32 ms.

Chunk (rows)	1	16	256	4096	32768
dynamic	33.87	10.09	8.55	8.40	**8.20**
guided	**8.14**	8.26	8.77	8.41	9.01

Dynamic needs chunks of at least about 256 rows to amortise scheduling. Guided performs well by default, because its chunks start large and shrink, and a large minimum chunk (32768) brings back tail imbalance.

9.6 Weak scaling (Gustafson)

With the work per thread fixed at 31,250 rows ($N = 31{,}250\,p$), ideal weak efficiency $T(1)/T(p)$ is 100%.

Table 18: Weak scaling, 31,250 rows per thread.

p	1	2	4	8	16
static (ms / efficiency)	2.42 / 100%	2.76 / 88%	3.01 / 81%	5.02 / 48%	9.33 / 26%
guided (ms / efficiency)	2.43 / 100%	2.48 / 98%	2.80 / 87%	4.75 / 51%	9.34 / 26%

Weak efficiency stays high up to 4 threads and falls once the combined bandwidth demand exceeds the memory system. This is the same ceiling that limits strong scaling, so for a bandwidth-bound kernel Gustafson's law gives no escape.

9.7 Roofline: multi-query cache blocking

One query performs 2 flops per 4-byte float read, which is 0.5 flop/byte. v2 adds `rf_topm_batch`: each row is loaded once and scored against Q queries while it is in L1. Inside, a register-blocked `dot4` scores four queries per row load. This raises arithmetic intensity to $0.5\,Q$ flop/byte. Each query keeps the exact summation order of the single-query dot product, so batched results are bit-identical, and the test suite checks this.

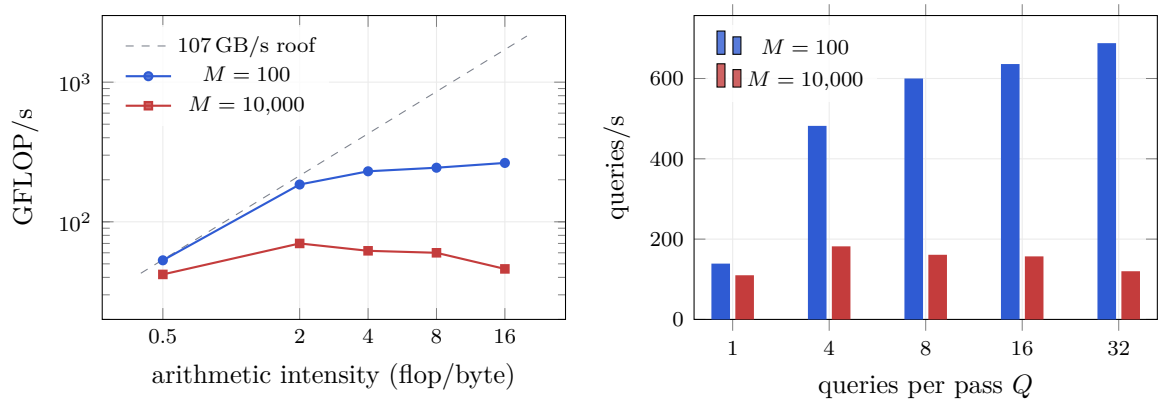


Figure 6: Multi-query blocking at 16 threads, $N = 500{,}000$. Left: attained performance against the measured bandwidth roof. Right: throughput.

With $M = 100$, throughput rises from 139 to 688 queries/s (4.9×). At $Q = 1$ the kernel sits on the bandwidth roof. From $Q \approx 4$ it leaves the roof and becomes compute- and heap-bound at about 250 GFLOP/s. With $M = 10{,}000$ the gain is small (1.65× at best): each thread keeps Q heaps of 10K entries, and heap maintenance dominates. Batching therefore suits high-volume evaluation with a modest M. The interactive path keeps one query and the methodology's $M = 10{,}000$.

9.8 Adaptive thread count and default schedule

The measurements also changed the product. On the live corpus (1,794 articles) v2's best is 4 threads guided (0.104 ms vs 0.152 ms serial), and 16 threads is slower than 4. Verification now chooses $p = \min(\text{cores}, 8, \lceil N/400 \rceil)$: enough rows per thread to amortise the fork/join and heap setup, and no more threads than the memory system can feed. On the live corpus this gives 5 threads. The default schedule is now guided. It was fastest on the live corpus, at 10K and at 50K. At 100K and 500K static was slightly faster in the strong-scaling sweep (8.04 vs 9.05 ms at 500K), but at 11 threads guided was faster (8.15 vs 9.14 ms, Table 16) and it avoids static's 11% idle time on the heterogeneous cores.

10 Threats to Validity and Limitations

1. **Headline as claim** on FakeNewsNet is a proxy, and it biases the task toward claims checkable from a headline.
2. **Syndication versus convergence.** Embedding similarity cannot separate syndication from independent outlets reporting the same wire fact, so the filter's measured effect is an upper bound.
3. **Dataset artefacts.** FEVER-trained models can key on claim wording, so FEVER accuracy alone would not show that retrieval does the work.
4. **Live corpus drift.** The corpus changes between runs, so timings are taken on a frozen snapshot. The benchmark reports the size it ran on.
5. **NLI behaviour on off-topic text.** Mitigated by the relevance floor, but the floor is a tuned threshold, and topically similar articles about different entities (e.g. a different train route) can still be labelled CONTRADICT.
6. **Heterogeneous cores and no pinning.** Results on Apple Silicon reflect the OS scheduler's placement across performance and efficiency cores. They are not directly comparable to pinned results on a homogeneous x86 server.

7. **Evaluation coverage.** Only LIAR has been run, against a corpus that does not cover its time span. FEVER and FakeNewsNet results remain future work.

11 Conclusion and Future Work

RealFact shows that evidence-based news verification over a live, multi-outlet corpus can run in about 100 ms per claim on a laptop CPU, with an exact retrieval kernel that scales to 4.61× on 500K articles and is bounded by memory bandwidth, as the roofline model predicts. The independence-aware aggregation stage does what the survey's discussion of conflicting and untrustworthy evidence asks for. In a controlled test it prevents ten copies of one report from flipping a contested claim to *Contradicted*, while leaving verdicts unchanged where there is no syndication. Each contribution is measured on its own axis, so either claim could have failed independently.

Applying the course's tools to the kernel showed where the time actually goes. Latency-bound single-thread code became bandwidth-bound after an ILP rewrite (2.6× faster serially). Karp–Flatt and weak scaling located the ceiling in shared memory bandwidth, not in serial code. Per-thread timing explained why guided beats static on heterogeneous cores. Raising arithmetic intensity through multi-query blocking was the one route past the bandwidth roof (5× throughput). Throughout, a 12-check suite kept every optimisation bit-exact against the serial build.

Future work.

- Index the FEVER Wikipedia dump and FakeNewsNet articles to obtain accuracy and Recall@K comparable to published baselines, and run the λ and τ sensitivity sweeps on hand-labelled syndication clusters.
- Runs on a pinned, homogeneous multi-socket x86 server to separate core heterogeneity and NUMA effects from scheduling effects.
- float16 or int8 embeddings, which halve or quarter the bytes per row and so raise the bandwidth-bound ceiling directly.
- Use the batched kernel inside the evaluation harness, and a heap-free top-M (threshold selection) to make batching pay off at large M.
- Compare against an ANN index as future work, since ANN changes what is retrieved.
- Generated justifications grounded in the cited clusters, covering the fourth stage of the survey's framework.

References

[1] Z. Guo, M. Schlichtkrull and A. Vlachos, "A survey on automated fact-checking," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 178–206, 2022. doi:10.1162/tacl_a_00454.

[2] J. Thorne, A. Vlachos, C. Christodoulopoulos and A. Mittal, "FEVER: a large-scale dataset for fact extraction and VERification," in *Proc. NAACL-HLT*, 2018, pp. 809–819.

[3] W. Y. Wang, "'Liar, liar pants on fire': A new benchmark dataset for fake news detection," in *Proc. ACL*, 2017, pp. 422–426.

[4] K. Shu, D. Mahudeswaran, S. Wang, D. Lee and H. Liu, "FakeNewsNet: A data repository with news content, social context, and spatiotemporal information for studying fake news on social media," *Big Data*, vol. 8, no. 3, pp. 171–188, 2020.

[5] S. Xiao, Z. Liu, P. Zhang and N. Muennighoff, "C-Pack: Packaged resources to advance general Chinese embedding," arXiv:2309.07597, 2023.

[6] OpenMP Architecture Review Board, *OpenMP Application Programming Interface, Version 5.2*, 2021.

[7] S. Williams, A. Waterman and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

A Reproducing the Results

```
make # build OpenMP + serial engine libraries
.venv/bin/python -m tests.test_correctness # 12-check correctness suite
./run.sh # start the app at http://127.0.0.1:5050
.venv/bin/python -m backend.bench --sizes 10000 50000 100000 500000
.venv/bin/python -m eval.run_eval --dataset liar \
    --path data/datasets/liar/test.tsv --limit 2000
```

Benchmark results are written to `data/bench_last.json` (from the UI) or `data/bench_cli.json` (from the command line). Evaluation results go to `data/eval_results.json`. The numbers in this report come from the v1 benchmark run finished at 2026-10-31 17:47:37, the v2 run finished at 2026-10-31 18:53:49 (saved in `data/bench_last.json`; the second v1 run is in `data/bench_v1_baseline.json`), and the LIAR run finished at 2026-10-31 17:50:12.